



Worksheet: Getting Started with Google Colab and PyTorch

This worksheet helps you understand the basics of Google Colab and PyTorch, including how to create tensors, perform operations, build simple neural networks, and visualize data using TensorBoard.

Learning Objectives:

- Understand how to use Google Colab to run Python code
- Create and inspect PyTorch tensors
- Perform basic tensor operations (shape, indexing, arithmetic, reshaping)
- Understand how PyTorch handles CPU/GPU devices

Preliminaries — Setup Environment

Google Colab (Google Colaboratory) is a cloud-based Jupyter notebook environment provided by Google. It allows you to:

- Run Python code directly in your web browser
- Use pre-installed scientific libraries (NumPy, PyTorch, etc.)
- Save notebooks to your Google Drive
- Share notebooks like Google Docs
- Access free GPU/TPU for accelerated computation

This makes Colab ideal for machine learning classes: students do not need to install anything locally, and training code runs quickly even on low-end laptops.

How to run code in Google Colab

A notebook is made up of cells. Each code cell can be executed independently.

- Click on a code cell
- Press Shift + Enter (or click the ► “Run” button to the left)

The output will appear directly below the cell.

Example:

```
In [ ]: print("Hello from Google Colab!")
```

Hello from Google Colab!

Enabling GPU in Google Colab

Google Colab allows you to run PyTorch code on a free GPU, but you must turn it on manually:

Steps to enable GPU

- At the top of the page, click: Runtime → Change runtime type
- In the popup window, find Hardware accelerator
- Select GPU
- Click Save

Colab will restart your notebook's environment. After this, your code will be able to access a GPU (if available).

You can verify that a GPU has been successfully enabled by running: *斜体文本*

```
In [ ]: !nvidia-smi
```

Mon Jan 19 16:21:00 2026

```
+-----+
+-----+
| NVIDIA-SMI 550.54.15              Driver Version: 550.54.15      CUDA Versio
n: 12.4      |
|-----+-----+
+-----+
| GPU   Name                     Persistence-M | Bus-Id        Disp.A | Volatile U
ncorr. ECC |
| Fan   Temp   Perf              Pwr:Usage/Cap |      Memory-Usage | GPU-Util
Compute M. |
|                               |                      |
MIG M. |
|=====+=====+=====+
=====|
|    0   Tesla T4                       Off | 00000000:00:04.0 Off |
0 |
| N/A    51C    P8                     9W / 70W |      0MiB / 15360MiB |      0%
Default |
|                               |                      |
N/A |
+-----+-----+
+-----+

+-----+
+-----+
| Processes:
|
| GPU   GI    CI          PID    Type    Process name
GPU Memory |
|       ID    ID
Usage      |
|=====+=====+=====+
=====|
| No running processes found
|
+-----+
+-----+
```

If you're running on Google Colab, PyTorch is pre-installed, but this cell demonstrates the typical setup.

```
In [ ]: !pip install -q torch torchvision

# Import core packages
import torch
import numpy as np

print("PyTorch Version:", torch.__version__)

# Detect device (GPU if available)
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

PyTorch Version: 2.9.0+cu126
Using device: cuda:0

Creating Basic PyTorch Tensors

We demonstrate the most common ways to create tensors:

- from Python lists
- zeros / ones
- random values

```
In [ ]: # From a Python list
a = torch.tensor([1, 2, 3])
print("Tensor a:", a)

# Zeros and ones
b = torch.zeros((2, 3))
c = torch.ones((3, 3))

print("Tensor b (zeros):\n", b)
print("Tensor c (ones):\n", c)

# Random tensor
d = torch.randn((2, 2))
print("Tensor d (random):\n", d)
```

```
Tensor a: tensor([1, 2, 3])
Tensor b (zeros):
tensor([[0., 0., 0.],
        [0., 0., 0.]])
Tensor c (ones):
tensor([[1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.]])
Tensor d (random):
tensor([[ 1.3934, -0.0218],
        [ 0.1091, -0.8452]])
```

Inspecting Tensors

We check:

- shape
- dtype (data type)
- device (CPU or GPU)

```
In [ ]: x = torch.randn(3, 4)

print("Tensor x:\n", x)
```

```
print("Shape:", x.shape)
print("Data type:", x.dtype)
print("Device:", x.device)
```

```
Tensor x:
  tensor([[ 1.1673, -0.3129, -0.1163,  0.5990],
          [ 0.0961, -0.0037, -0.1511, -2.2460],
          [-0.3059, -1.0267, -0.7473,  0.8252]])
Shape: torch.Size([3, 4])
Data type: torch.float32
Device: cpu
```

Basic Tensor Operations

This section demonstrates fundamental operations you will use frequently when working with PyTorch tensors:

- Arithmetic (addition, multiplication, broadcast operations)
- Indexing / slicing (accessing parts of a tensor)
- Reshaping (changing tensor dimensions)
- Concatenation & stacking (combining tensors)
- Reduction operations (min, max, mean, sum)

Arithmetic

```
In [ ]: x = torch.tensor([1., 2., 3.])
        y = torch.tensor([10., 20., 30.])

        print("x + y =", x + y)
        print("x * y =", x * y)
        print("x ** 2 =", x ** 2) # square each element

        # Broadcasting (automatically expanding dimensions)
        m = torch.ones((3, 3))
        v = torch.tensor([1., 2., 3.])
        print("\nBroadcasting example: m + v =", m + v)
```

```
x + y = tensor([11., 22., 33.])
x * y = tensor([10., 40., 90.])
x ** 2 = tensor([1., 4., 9.])
```

```
Broadcasting example: m + v =
  tensor([[2., 3., 4.],
          [2., 3., 4.],
          [2., 3., 4.]])
```

Indexing & Slicing

```
In [ ]: z = torch.arange(10) # [0,1,2,...,9]
```

```
print("z:", z)
print("z[0]:", z[0])      # first element
print("z[-1]:", z[-1])   # last element
print("z[2:6]:", z[2:6])  # slice
print("z[::2]:", z[::2])  # every 2 steps

mat = torch.arange(12).reshape(3, 4)
print("Matrix:\n", mat)

print("First row:", mat[0])
print("Column 2:", mat[:, 2])
print("Submatrix (rows 1-2, columns 1-3):\n", mat[1:3, 1:4])
```

```
z: tensor([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
z[0]: tensor(0)
```

```
z[-1]: tensor(9)
```

```
z[2:6]: tensor([2, 3, 4, 5])
```

```
z[::2]: tensor([0, 2, 4, 6, 8])
```

```
Matrix:
```

```
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

```
First row: tensor([0, 1, 2, 3])
```

```
Column 2: tensor([ 2,  6, 10])
```

```
Submatrix (rows 1-2, columns 1-3):
```

```
tensor([[ 5,  6,  7],
        [ 9, 10, 11]])
```

Reshaping

```
In [ ]: x = torch.arange(12)
```

```
print("Original:", x)
print("Reshaped to 3x4:\n", x.reshape(3, 4))
print("Reshaped to 2x2x3:\n", x.reshape(2, 2, 3))

print("Using view (same shape 3x4):\n", x.view(3, 4))
```

Original: tensor([0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11])

Reshaped to 3x4:

```
tensor([[ 0, 1, 2, 3],
        [ 4, 5, 6, 7],
        [ 8, 9, 10, 11]])
```

Reshaped to 2x2x3:

```
tensor([[[ 0, 1, 2],
         [ 3, 4, 5]],
```

```
        [[ 6, 7, 8],
         [ 9, 10, 11]])]
```

Using view (same shape 3x4):

```
tensor([[ 0, 1, 2, 3],
        [ 4, 5, 6, 7],
        [ 8, 9, 10, 11]])
```

Concatenation & Stacking

```
In [ ]: a = torch.tensor([1, 2, 3])
        b = torch.tensor([4, 5, 6])

        print("Concatenate:", torch.cat([a, b], dim=0)) # [1,2,3,4,5,6]
        print("Stack (creates a new dimension):\n", torch.stack([a, b], dim=0))
```

Concatenate: tensor([1, 2, 3, 4, 5, 6])

Stack (creates a new dimension):

```
tensor([[1, 2, 3],
        [4, 5, 6]])
```

Reduction Operations

```
In [ ]: x = torch.randn(3, 4)

        print("x:\n", x)
        print("Sum:", x.sum())
        print("Mean:", x.mean())
        print("Max:", x.max())
        print("Min:", x.min())

        # Reduce along specific dimensions
        print("\nColumn-wise mean:", x.mean(dim=0))
        print("Row-wise sum:", x.sum(dim=1))
```

x:

```
tensor([[ 0.0935, -0.2793, -1.2905,  1.2861],
        [ 0.4826,  0.0718,  0.2356, -0.5360],
        [-0.1700,  0.2678,  1.7992, -0.0950]])
```

Sum: tensor(1.8658)

Mean: tensor(0.1555)

Max: tensor(1.7992)

Min: tensor(-1.2905)

Column-wise mean: tensor([0.1354, 0.0201, 0.2481, 0.2184])

Row-wise sum: tensor([-0.1903, 0.2540, 1.8021])

Moving Tensors Between CPU and GPU

Only works if GPU is available (Colab usually provides one).

```
In [ ]: x_cpu = torch.ones((2, 2))
x_gpu = x_cpu.to(device)    # move to GPU (if available)

print("CPU tensor:", x_cpu)
print("GPU/Device tensor:", x_gpu)
print("Stored on:", x_gpu.device)
```

```
CPU tensor: tensor([[1., 1.],
                   [1., 1.]])
GPU/Device tensor: tensor([[1., 1.],
                           [1., 1.]], device='cuda:0')
Stored on: cuda:0
```